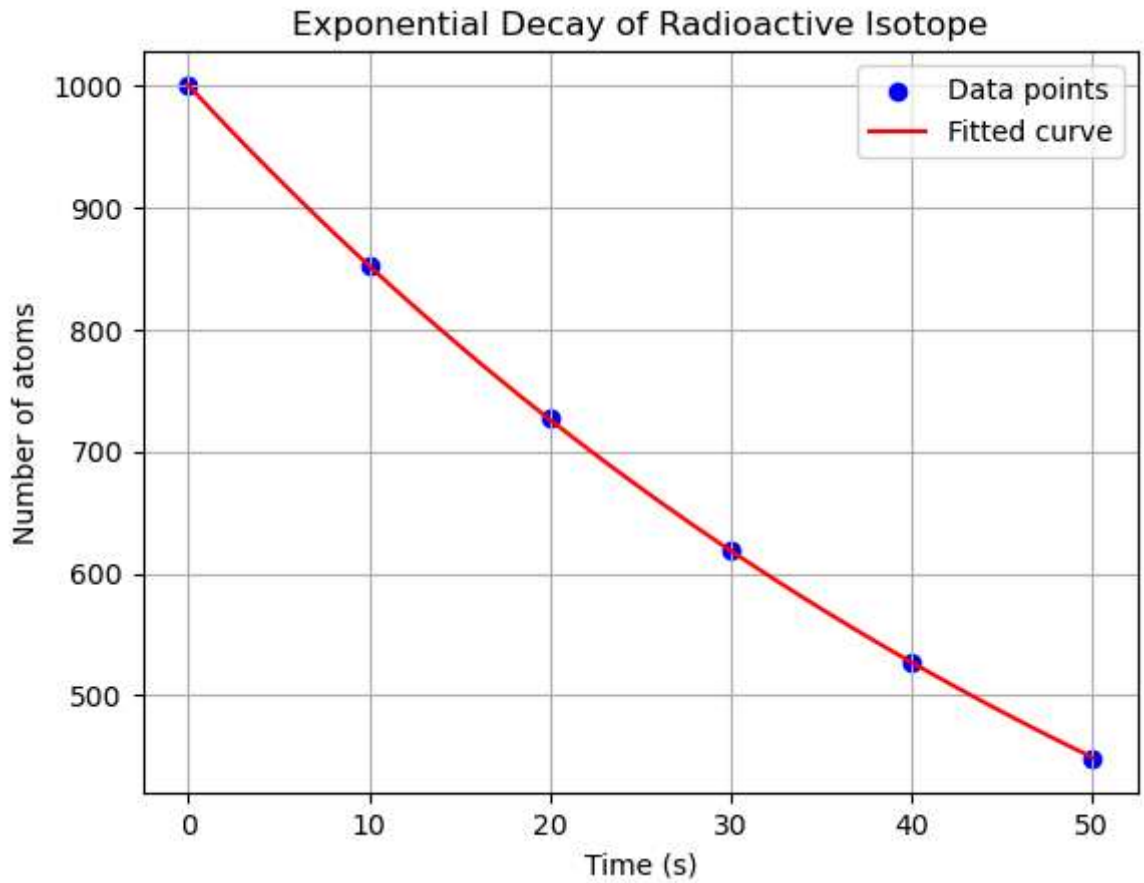
In [1]:
```
# Q1 (a)
```

In [6]:
```
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# Time and number of atoms data
t = np.array([0, 10, 20, 30, 40, 50])
N = np.array([1000, 853, 727, 619, 527, 448])

# Exponential decay function
def exp_decay(t, N0, lambda_):
    return N0 * np.exp(-lambda_ * t)

# Fit the model to data, using initial guess for N0 and lambda
params, _ = curve_fit(exp_decay, t, N, p0=[1000, 0.02])
N0_fit, lambda_fit = params  # Get best fit values

# Print fitted values in a simple way
print("Initial number of atoms (N0):", round(N0_fit, 2))
print("Decay constant (lambda):", round(lambda_fit, 4))

# Make smooth curve for fitted line
t_fit = np.linspace(0, 50, 100)
N_fit = exp_decay(t_fit, N0_fit, lambda_fit)

# Plot the data and the fitted curve
plt.scatter(t, N, color='blue', label='Data points')
plt.plot(t_fit, N_fit, color='red', label='Fitted curve')
plt.xlabel('Time (s)')
plt.ylabel('Number of atoms')
plt.title('Exponential Decay of Radioactive Isotope')
plt.legend()
plt.grid(True)
plt.show()
```

```
Initial number of atoms (N0): 1000.75
Decay constant (lambda): 0.016
```



In [7]:
```
# Q1. (b)
```

In [11]:
```
# Predict values using the fitted model
N_pred = exp_decay(t, N0_fit, lambda_fit)

# Calculate R-squared
SS_res = np.sum((N - N_pred) ** 2)            # Residual sum of squares
SS_tot = np.sum((N - np.mean(N)) ** 2)        # Total sum of squares
R_squared = 1 - (SS_res / SS_tot)

print("R-squared:", round(R_squared, 4))

'''
R^2 ranges from 0 to 1:
1.0 = perfect fit.
0.0 = the model explains nothing
'''
```

```
R-squared: 1.0
```

Out[11]:
```
'\nR^2 ranges from 0 to 1:\n1.0 = perfect fit.\n0.0 = the model explains nothing
\n'
```

In [ ]: